

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Experiments

Code of Conduct:

I D. Sindhu (192424197) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

D. Sindhu.

Signature of the student:

Annexure A

Short Answer Test

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

6. Design and conduct an experiment to implement a **Decision Tree classifier** on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

7. Perform an experiment using the **K-Nearest Neighbors (KNN)** algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan)

to observe their effect on model performance. Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries.

8. Implement a **Support Vector Machine (SVM)** model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

9. Design an experiment to implement a **clustering algorithm such as K-Means** on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

10. Conduct an experiment using a **logistic regression model** for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

11. Design and conduct an experiment to implement a **Random Forest classifier** on a real-world dataset. Train the model with varying numbers of trees and maximum depth. Evaluate performance using accuracy, precision, recall, and F1-score. Analyze how ensemble size and feature randomness affect overfitting and generalization, and compare results with a single decision tree.

12. Perform an experiment using a **Principal Component Analysis (PCA)** technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyze how variance retention influences information loss and classification results.

13. Implement a **Gradient Descent optimization algorithm** and conduct experiments by varying learning rates and initialization values. Observe convergence behavior and record the number of iterations required to reach optimal values. Analyze cases of slow convergence or divergence, and suggest improvements such as momentum or adaptive learning rate methods.

14. Design an experiment to implement a **Neural Network classifier** for a classification task. Vary the number of hidden layers, neurons, and activation functions. Evaluate the model using accuracy and loss curves, and analyze how network architecture affects performance, convergence, and overfitting.

15. Conduct an experiment using a **hierarchical clustering algorithm** on an unlabeled dataset. Compare different linkage methods such as single, complete, and average linkage. Evaluate clustering quality using dendrogram analysis and silhouette score. Analyze how linkage criteria influence cluster formation and interpretability of results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Experiment 1: Naïve Bayes Classifier

Aim

To implement a Naïve Bayes classifier using a real-world dataset and evaluate its performance.

Procedure

1. Import the required libraries.
2. Load the Breast Cancer dataset.
3. Split the dataset into training and testing sets.
4. Train the Naïve Bayes classifier.
5. Predict the test data.
6. Calculate accuracy, precision, and recall.
7. Display the results.

Program

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42

```

```
)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall  :", recall_score(y_test, y_pred))
```

Sample Input

- **Dataset:** Breast Cancer Wisconsin Dataset
- **Training Data:** 70%
- **Testing Data:** 30%

Sample Output

Accuracy: 0.94
Precision: 0.95
Recall: 0.96

Result

The Naïve Bayes classifier was successfully implemented. The model achieved good accuracy, precision, and recall on the Breast Cancer dataset.

Experiment 2: Maximum Likelihood Estimation (MLE) vs Bayesian Estimation

Aim

To estimate the mean and variance of a dataset using **Maximum Likelihood Estimation (MLE)** and compare the results with **Bayesian Estimation**.

Procedure

1. Import the required libraries.
2. Create a sample dataset.
3. Calculate the mean and variance using MLE.
4. Estimate the mean using Bayesian Estimation.
5. Compare the results.
6. Display the estimated values.

Program

```
import numpy as np
data = np.array([10, 12, 11, 13, 15, 14, 16, 12, 11, 13])
mle_mean = np.mean(data)
mle_variance = np.var(data)
prior_mean = 12
bayesian_mean = (prior_mean + mle_mean) / 2
print("MLE Mean:", mle_mean)
print("MLE Variance:", mle_variance)
print("Bayesian Mean:", bayesian_mean)
```

Sample Input

Dataset:
10, 12, 11, 13, 15, 14, 16, 12, 11, 13

Sample Output

MLE Mean: 12.7

MLE Variance: 3.21

Bayesian Mean: 12.35

Result

The parameters were successfully estimated using **Maximum Likelihood Estimation (MLE)** and **Bayesian Estimation**. MLE used only the sample data, while Bayesian Estimation combined the sample data with prior information, resulting in a slightly different estimated mean.

Experiment 3: Expectation-Maximization (EM) Algorithm

Aim

To implement the **Expectation-Maximization (EM)** algorithm using a Gaussian Mixture Model and observe its clustering performance.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Train the Gaussian Mixture Model (EM algorithm).
4. Predict the cluster labels.
5. Display the predicted clusters.

Program

```
from sklearn.datasets import load_iris
from sklearn.mixture import GaussianMixture
iris = load_iris()
X = iris.data
model = GaussianMixture(n_components=3, random_state=42)
model.fit(X)
clusters = model.predict(X)
print("Predicted Clusters:")
print(clusters)
```

Sample Input

- **Dataset:** Iris Dataset
- **Number of Clusters:** 3

Sample Output

Predicted Clusters:

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 ...
 2 2 2 2 2 2 ...
 0 0 0 0 0 0 ...]
```

Result

The EM algorithm was successfully implemented using the Gaussian Mixture Model. The model grouped the Iris dataset into **3 clusters**, demonstrating effective probabilistic clustering.

Experiment 4: Bayesian Belief Network (BBN)

Aim

To construct a **Bayesian Belief Network (BBN)** and perform inference using given probabilities.

Procedure

1. Import the required library.
2. Create a Bayesian Belief Network.
3. Define the conditional probability tables (CPTs).
4. Perform inference using evidence.
5. Display the posterior probability.

Program

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = DiscreteBayesianNetwork([
    ('Flu', 'Fever'),
    ('Flu', 'Cough')
])
cpd_flu = TabularCPD(
    variable='Flu',
    variable_card=2,
    values=[[0.75], [0.25]]
)
cpd_fever = TabularCPD(
    variable='Fever',
    variable_card=2,
    values=[[0.9, 0.2],
            [0.1, 0.8]],
    evidence=['Flu'],
    evidence_card=[2]
)
cpd_cough = TabularCPD(
    variable='Cough',
    variable_card=2,
    values=[[0.8, 0.3],
            [0.2, 0.7]],
    evidence=['Flu'],
    evidence_card=[2]
)
model.add_cpds(cpd_flu, cpd_fever, cpd_cough)
inference = VariableElimination(model)
result = inference.query(variables=['Flu'], evidence={'Fever': 1})
print(result)
```

Sample Input

- **Evidence:** Fever = Yes

Sample Output

Flu	Probability
No	0.27
Yes	0.73

Result

The Bayesian Belief Network was successfully implemented. The posterior probability of **Flu** was calculated based on the given evidence (**Fever = Yes**), showing how Bayesian inference handles uncertainty in probabilistic reasoning.

Experiment 5: Candidate Elimination Algorithm

Aim

To implement the **Candidate Elimination Algorithm** and identify the specific and general hypotheses from the given training data.

Procedure

1. Import the required library.
2. Create the training dataset.
3. Initialize the specific and general hypotheses.
4. Update the hypotheses using the Candidate Elimination algorithm.
5. Display the final specific and general hypotheses.

Program

```
import pandas as pd
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]
df = pd.DataFrame(data, columns=[
    'Sky', 'AirTemp', 'Humidity', 'Wind',
    'Water', 'Forecast', 'EnjoySport'
])
specific = df.iloc[0, :-1].tolist()
general = [['?' for _ in range(len(specific))] for _ in range(len(specific))]
for i, row in df.iterrows():
    if row['EnjoySport'] == 'Yes':
        for j in range(len(specific)):
            if specific[j] != row[j]:
                specific[j] = '?'
                general[j][j] = '?'
    else:
        for j in range(len(specific)):
            if specific[j] != row[j]:
                general[j][j] = specific[j]
            else:
                general[j][j] = '?'
```



```

print("Specific Hypothesis:")
print(specific)
print("\nGeneral Hypothesis:")
for g in general:
    print(g)

```

Sample Input

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Sample Output

Specific Hypothesis:

```
['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

General Hypothesis:

```
['Sunny', '?', '?', '?', '?', '?']
```

```
['?', 'Warm', '?', '?', '?', '?']
```

```
['?', '?', '?', '?', '?', '?']
```

```
['?', '?', '?', '?', '?', '?']
```

```
['?', '?', '?', '?', '?', '?']
```

```
['?', '?', '?', '?', '?', '?']
```

Result

The Candidate Elimination algorithm was successfully implemented. The final **specific hypothesis** and **general hypothesis** were obtained based on the training examples, representing the version space consistent with the given data.

Experiment 6: Decision Tree Classifier

Aim

To implement a **Decision Tree classifier** on a real-world dataset and evaluate its performance using **accuracy** and **F1-score**.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Train the Decision Tree classifier using the Gini criterion.
5. Predict the test data.
6. Calculate the Accuracy and F1-score.
7. Display the results.

Program

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

```

```

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred, average='weighted'))

```

Sample Input

- **Dataset:** Iris Dataset
- **Training Data:** 70%
- **Testing Data:** 30%

Sample Output

Accuracy : 1.0
F1 Score : 1.0

Result

The Decision Tree classifier was successfully implemented using the **Iris dataset**. The model achieved high **Accuracy** and **F1-score**, demonstrating effective classification performance.

Experiment 7: K-Nearest Neighbours (KNN) Classifier

Aim

To implement the **K-Nearest Neighbours (KNN)** algorithm on a classification dataset and evaluate its performance using **accuracy** and a **confusion matrix**.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Train the KNN classifier with **K = 3**.
5. Predict the test data.
6. Calculate the accuracy.
7. Display the confusion matrix and results.

Program

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

```

```
)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Sample Input

- **Dataset:** Iris Dataset
- **K Value:** 3
- **Training Data:** 70%
- **Testing Data:** 30%

Sample Output

```
Accuracy : 1.0
Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Result

The **K-Nearest Neighbours (KNN)** classifier was successfully implemented on the Iris dataset. The model achieved high classification accuracy, and the confusion matrix showed correct prediction of most test samples.

Experiment 8: Support Vector Machine (SVM) Classifier

Aim

To implement a **Support Vector Machine (SVM)** classifier on a classification dataset and evaluate its performance using **accuracy** and **precision**.

Procedure

1. Import the required libraries.
2. Load the Breast Cancer dataset.
3. Split the dataset into training and testing sets.
4. Train the SVM classifier using the **RBF kernel**.
5. Predict the test data.
6. Calculate the accuracy and precision.
7. Display the results.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score
data = load_breast_cancer()
X = data.data
y = data.target
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = SVC(kernel='rbf', random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))

```

Sample Input

- **Dataset:** Breast Cancer Wisconsin Dataset
- **Kernel:** RBF
- **Training Data:** 70%
- **Testing Data:** 30%

Sample Output

Accuracy : 0.96

Precision: 0.95

Result

The **Support Vector Machine (SVM)** classifier was successfully implemented using the **RBF kernel**. The model achieved high **accuracy** and **precision**, showing effective classification performance on the Breast Cancer dataset.

Experiment 9: K-Means Clustering

Aim

To implement the **K-Means clustering** algorithm on an unlabeled dataset and evaluate its performance using **inertia** and **silhouette score**.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Train the K-Means model with **3 clusters**.
4. Predict the cluster labels.
5. Calculate the inertia and silhouette score.
6. Display the results.

Program

```

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
iris = load_iris()
X = iris.data
model = KMeans(n_clusters=3, random_state=42)
model.fit(X)
labels = model.labels_
print("Inertia :", model.inertia_)
print("Silhouette Score :", silhouette_score(X, labels))

```

Sample Input

- **Dataset:** Iris Dataset
- **Number of Clusters:** 3

Sample Output

Inertia : 78.85

Silhouette Score : 0.55

Result

The **K-Means clustering** algorithm was successfully implemented on the Iris dataset. The model formed **3 clusters**, and the inertia and silhouette score indicated good clustering performance.

Experiment 10: Logistic Regression

Aim

To implement a **Logistic Regression** model for binary classification and evaluate its performance using **accuracy, precision, and recall**.

Procedure

1. Import the required libraries.
2. Load the Breast Cancer dataset.
3. Split the dataset into training and testing sets.
4. Train the Logistic Regression model.
5. Predict the test data.
6. Calculate accuracy, precision, and recall.
7. Display the results.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall  :", recall_score(y_test, y_pred))
```

Sample Input

- **Dataset:** Breast Cancer Wisconsin Dataset
- **Training Data:** 70%
- **Testing Data:** 30%

Sample Output

Accuracy : 0.98

Precision: 0.98

Recall : 0.98

Result

The **Logistic Regression** model was successfully implemented on the Breast Cancer dataset. The model achieved high **accuracy, precision, and recall**, demonstrating effective binary classification.

Experiment 11: Random Forest Classifier

Aim

To implement a **Random Forest classifier** on a real-world dataset and evaluate its performance using **accuracy, precision, recall, and F1-score**.

Procedure

1. Import the required libraries.
2. Load the Breast Cancer dataset.
3. Split the dataset into training and testing sets.
4. Train the Random Forest classifier.
5. Predict the test data.
6. Calculate accuracy, precision, recall, and F1-score.
7. Display the results.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

Sample Input

- **Dataset:** Breast Cancer Wisconsin Dataset
- **Number of Trees:** 100
- **Training Data:** 70%
- **Testing Data:** 30%

Sample Output

Accuracy : 0.97
Precision: 0.98
Recall : 0.97
F1 Score : 0.97

Result

The **Random Forest classifier** was successfully implemented on the Breast Cancer dataset. The model achieved high **accuracy, precision, recall, and F1-score**, demonstrating effective classification performance.

Experiment 12: Principal Component Analysis (PCA)

Aim

To implement **Principal Component Analysis (PCA)** for dimensionality reduction and evaluate its effect on classification accuracy.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Apply PCA to reduce the features to 2 principal components.
4. Split the dataset into training and testing sets.
5. Train a Logistic Regression classifier.
6. Calculate the accuracy.
7. Display the results.

Program

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
iris = load_iris()
X = iris.data
y = iris.target
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.3, random_state=42
)
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Explained Variance Ratio :", pca.explained_variance_ratio_)
```

Sample Input

- **Dataset:** Iris Dataset
- **Principal Components:** 2

Sample Output

Accuracy : 0.98

Explained Variance Ratio :

[0.9246 0.0531]

Output values may vary slightly depending on the train-test split.

Result

The **Principal Component Analysis (PCA)** technique was successfully implemented to reduce the dataset from **4 features to 2 principal components**. The classifier maintained high accuracy while reducing the number of features, demonstrating effective dimensionality reduction.

Experiment 13: Gradient Descent

Aim

To implement the **Gradient Descent** optimization algorithm and observe its convergence for finding the minimum value of a function.

Procedure

1. Import the required library.
2. Define a simple function ($f(x)=x^2$).
3. Initialize the starting value and learning rate.
4. Apply the Gradient Descent algorithm.
5. Update the value of (x) in each iteration.
6. Display the optimized value and function value.

Program

```
x = 10
learning_rate = 0.1
iterations = 50
for i in range(iterations):
    gradient = 2 * x
    x = x - learning_rate * gradient
print("Optimized x =", x)
print("Minimum value =", x**2)
```

Sample Input

Initial Value = 10

Learning Rate = 0.1

Iterations = 50

Sample Output

Optimized x = 0.000143

Minimum value = 2.04e-08

Result

The **Gradient Descent** algorithm was successfully implemented. The value of x gradually converged to **0**, minimizing the function ($f(x)=x^2$). This demonstrates how Gradient Descent optimizes a function by updating parameters iteratively.

Experiment 14: Neural Network Classifier (Simple)

Aim

To implement a **Neural Network classifier** for a classification task and evaluate its performance using **accuracy**.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Train the Neural Network classifier.
5. Predict the test data.
6. Calculate the accuracy.
7. Display the results.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
```

Sample Input

- **Dataset:** Iris Dataset
- **Hidden Layer:** 10 Neurons
- **Training Data:** 70%
- **Testing Data:** 30%
-

Sample Output

Accuracy : 0.98

Result

The **Neural Network classifier** was successfully implemented on the Iris dataset. The model achieved high **accuracy**, demonstrating effective classification performance.

Experiment 15: Hierarchical Clustering

Aim

To implement **Hierarchical Clustering** on an unlabeled dataset and evaluate the clustering performance using the **silhouette score**.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Apply Hierarchical Clustering with **3 clusters**.
4. Predict the cluster labels.
5. Calculate the silhouette score.
6. Display the results.

Program

```
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
iris = load_iris()
X = iris.data
model = AgglomerativeClustering(n_clusters=3)
labels = model.fit_predict(X)
print("Silhouette Score :", silhouette_score(X, labels))
print("Cluster Labels:")
print(labels)
```

Sample Input

- **Dataset:** Iris Dataset
- **Number of Clusters:** 3

Sample Output

Silhouette Score : 0.55

Cluster Labels:

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 ...
 0 0 0 0 0 ...
 2 2 2 2 2 ...]
```

Result

The **Hierarchical Clustering** algorithm was successfully implemented on the Iris dataset.

The data was grouped into **3 clusters**, and the silhouette score indicated satisfactory clustering quality.